

HR Management System — Regression Test Case Suite

Document ID: HRMS-RTC-001

Version: 1.0

Prepared by: QA Team

Review status: Approved

Framework: NestJS 11 · PostgreSQL 15 · Prisma ORM · Multi-tenant SaaS

Test types covered: Functional · Security · Boundary · Negative · Integration

Document conventions

Field	Description
Test Case ID	Module prefix + sequential number (e.g., AUTH-001)
Priority	P0 = Blocker · P1 = Critical · P2 = Major · P3 = Minor
Severity	Critical · High · Medium · Low
Type	Functional · Security · Boundary · Negative · Integration · Performance
Phase	V1 · V2 · Both
Status	Active · Deprecated · Blocked
Expected result	The exact observable outcome when the test passes
Actual result	Filled by tester during execution
Pass/Fail	Filled by tester during execution

Module 1 — Authentication & Security

AUTH-001

Field	Value
Test Case ID	AUTH-001
Title	Successful login with valid credentials

Field	Value
Module	Authentication
Priority	P0
Severity	Critical
Type	Functional
Phase	V1
Status	Active

Preconditions:

- A registered and email-verified user account exists with email `test.user@acme.com` and a known password
- The company account is active (`is_active = TRUE`)

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /auth/login</code> with <code>{ "email": "test.user@acme.com", "password": "ValidPass@123" }</code>	HTTP 200 response
2	Inspect response body	Contains <code>accessToken</code> (JWT), <code>expiresIn: 900</code> , and <code>tokenType: "Bearer"</code>
3	Inspect <code>Set-Cookie</code> response header	Contains <code>refresh_token</code> cookie with <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Strict</code> flags
4	Decode the JWT payload	Contains <code>sub</code> (userId), <code>companyId</code> , <code>roles[]</code> , <code>iat</code> , <code>exp</code> ; <code>exp - iat = 900</code>
5	Inspect <code>users.last_login_at</code> in database	Updated to the current timestamp (within 5 seconds)

Expected Result: HTTP 200; access token issued; refresh token set in HttpOnly cookie; `last_login_at` updated in DB.

Actual Result: _____

Pass / Fail: _____

Notes: _____

AUTH-002

Field	Value
Test Case ID	AUTH-002
Title	Login rejected with incorrect password
Module	Authentication
Priority	P0
Severity	Critical
Type	Negative
Phase	V1
Status	Active

Preconditions:

- Valid user account exists for `test.user@acme.com`

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /auth/login</code> with <code>{ "email": "test.user@acme.com", "password": "WrongPassword" }</code>	HTTP 401 response
2	Inspect response body	<code>{ "error": "INVALID_CREDENTIALS", "message": "Invalid email or password" }</code> — must NOT reveal which field is wrong
3	Verify no <code>Set-Cookie</code> header is present	No cookie issued
4	Verify <code>users.last_login_at</code> in database	NOT updated (remains previous value)

Expected Result: HTTP 401; generic error message; no tokens issued; last_login_at unchanged.

Actual Result: _____

Pass / Fail: _____

AUTH-003

Field	Value
Test Case ID	AUTH-003
Title	Refresh token rotation — old token invalidated after use
Module	Authentication
Priority	P0
Severity	Critical
Type	Security
Phase	V1
Status	Active

Preconditions:

- User is logged in; valid refresh token `RT-1` is stored in cookie

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /auth/refresh</code> with cookie <code>RT-1</code>	HTTP 200; new access token issued; new refresh token <code>RT-2</code> set in cookie
2	Verify in Redis	<code>RT-1</code> hash is revoked (<code>revoked_at</code> set); <code>RT-2</code> hash is active
3	Send <code>POST /auth/refresh</code> again with the old cookie <code>RT-1</code>	HTTP 401 — token already revoked
4	Inspect response body	<code>{ "error": "TOKEN_REVOKED", "message": "Refresh token is invalid or has been revoked" }</code>

Expected Result: RT-1 revoked after first use; reuse of RT-1 returns 401.

Actual Result: _____

Pass / Fail: _____

AUTH-004

Field	Value
Test Case ID	AUTH-004
Title	JWT access token rejected after expiry
Module	Authentication
Priority	P0
Severity	Critical
Type	Security
Phase	V1
Status	Active

Preconditions:

- An expired JWT (15 minutes old) is available

Test Steps:

Step	Action	Expected Result
1	Send GET /employees with Authorization: Bearer <expired-token>	HTTP 401
2	Inspect response body	{ "error": "TOKEN_EXPIRED", "message": "Access token has expired" }
3	Send GET /employees with no Authorization header	HTTP 401 with { "error": "UNAUTHORIZED" }
4	Send GET /employees with a malformed token "Bearer not.a.jwt"	HTTP 401 with { "error": "INVALID_TOKEN" }

Expected Result: All three invalid token scenarios return HTTP 401 with appropriate error codes.

Actual Result: _____

Pass / Fail: _____

AUTH-005

Field	Value
Test Case ID	AUTH-005
Title	Cross-tenant data isolation — company A cannot access company B data
Module	Authentication / Multi-tenancy
Priority	P0
Severity	Critical
Type	Security
Phase	V1
Status	Active

Preconditions:

- Two separate companies exist: Company A (companyId = co-A) and Company B (companyId = co-B)
- Employee (emp-B-1) belongs to Company B
- User from Company A is authenticated with a valid JWT containing (companyId = co-A)

Test Steps:

Step	Action	Expected Result
1	Send <code>GET /employees/emp-B-1</code> using Company A's access token	HTTP 404 — not found (not 403, to avoid confirming existence)
2	Send <code>PATCH /employees/emp-B-1</code> using Company A's access token	HTTP 404
3	Send <code>GET /employees</code> using Company A's access token	Returns only Company A employees; no Company B employees in results
4	Verify in database	<code>SELECT * FROM employees WHERE id = 'emp-B-1'</code> returns a row — confirms record exists but was correctly hidden

Expected Result: Company A JWT cannot read, modify, or detect Company B resources.

Actual Result: _____

Pass / Fail: _____

AUTH-006

Field	Value
Test Case ID	AUTH-006
Title	Rate limiting blocks brute-force login attempts
Module	Authentication
Priority	P1
Severity	High
Type	Security
Phase	V1
Status	Active

Preconditions:

- Rate limit configured: 10 failed login attempts per IP per minute triggers a logout

Test Steps:

Step	Action	Expected Result
1	Send 10 consecutive <code>POST /auth/login</code> requests with wrong password from the same IP	First 10 return HTTP 401
2	Send the 11th request from the same IP (even with the correct password)	HTTP 429 <code>Too Many Requests</code>
3	Inspect response headers on the 429	Contains <code>Retry-After</code> header with seconds until reset
4	Wait for the lockout window to expire, then send a valid login	HTTP 200 — access restored

Expected Result: 429 after threshold; Retry-After header present; access restored after window.

Actual Result: _____

Pass / Fail: _____

AUTH-007

Field	Value
Test Case ID	AUTH-007
Title	RBAC — Employee role blocked from accessing HR-only endpoints
Module	RBAC
Priority	P0
Severity	Critical
Type	Security
Phase	V1
Status	Active

Preconditions:

- User `emp-user` has role `Employee` only
- User `hr-user` has role `HR Manager`

Test Steps:

Step	Action	Expected Result
1	Send <code>GET /employees</code> using <code>emp-user</code> token	HTTP 403 — insufficient permissions
2	Send <code>POST /employees</code> using <code>emp-user</code> token	HTTP 403
3	Send <code>GET /payroll/cycles</code> using <code>emp-user</code> token	HTTP 403
4	Send <code>GET /employees</code> using <code>hr-user</code> token	HTTP 200 — returns employee list
5	Send <code>GET /auth/me</code> using <code>emp-user</code> token	HTTP 200 — employee can see their own profile
6	Send <code>GET /payslips</code> using <code>emp-user</code> token	HTTP 200 — employee can see their own payslips

Expected Result: Employee role restricted to self-scope endpoints; HR Manager has company-scope read access.

Actual Result: _____

Pass / Fail: _____

AUTH-008

Field	Value
Test Case ID	AUTH-008
Title	Password reset token is single-use and expires after 1 hour
Module	Authentication
Priority	P1
Severity	High
Type	Security
Phase	V1
Status	Active

Preconditions:

- User exists; password reset email triggered; reset token (T1) extracted

Test Steps:

Step	Action	Expected Result
1	Send (POST /auth/reset-password) with token (T1) and new password (NewPass@456)	HTTP 200 — password changed
2	Verify login with new password	HTTP 200 — login successful
3	Send (POST /auth/reset-password) again with the same token (T1)	HTTP 400 — { "error": "TOKEN_ALREADY_USED" }
4	Manually set (password_reset_expires) to 2 hours ago in DB; send reset request with a fresh token	HTTP 400 — { "error": "TOKEN_EXPIRED" }

Expected Result: Reset token is single-use; expired tokens are rejected.

Actual Result: _____

Pass / Fail: _____

Module 2 — Employee Management

EMP-001

Field	Value
Test Case ID	EMP-001
Title	Create employee with all required fields
Module	Employee Management
Priority	P0
Severity	Critical
Type	Functional
Phase	V1
Status	Active

Preconditions:

- HR Manager is authenticated
- Department (dept-eng), JobTitle (jt-swe), Location (loc-dhaka) exist in the company

Test Steps:

Step	Action	Expected Result
1	Send (POST /employees) with valid payload: (firstName), (lastName), (email), (hireDate), (departmentId), (jobTitleId), (employmentType: "full_time")	HTTP 201
2	Inspect response body	Employee object returned with auto-generated (id), (employeeNumber), (status: "active"), (createdAt)
3	Verify in (employees) table	Row exists with correct (company_id), all submitted fields saved, (deleted_at = NULL)
4	Verify in (employment_history) table	One row with (event_type = "hire"), correct (effective_date = hireDate)
5	Verify in (audit_logs) table	One row with (action = "create"), (resource = "employee"), (resource_id = <new employee id>), (user_id = <HR user id>)

Expected Result: Employee created; hire event in history; audit log entry created.

Actual Result: _____

Pass / Fail: _____

EMP-002

Field	Value
Test Case ID	EMP-002
Title	Duplicate employee number rejected
Module	Employee Management
Priority	P0

Field	Value
Severity	High
Type	Negative
Phase	V1
Status	Active

Preconditions:

- Employee with `employee_number = "EMP-001"` already exists in the company

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /employees</code> with <code>employeeNumber: "EMP-001"</code> and all other valid fields	HTTP 409
2	Inspect response body	<code>{ "error": "DUPLICATE_EMPLOYEE_NUMBER", "message": "Employee number EMP-001 is already in use" }</code>
3	Verify in <code>employees</code> table	No new row created
4	Verify in <code>audit_logs</code> table	<code>action = "attempt_failed"</code> entry logged

Expected Result: HTTP 409; no duplicate created; failed attempt logged.

Actual Result: _____

Pass / Fail: _____

EMP-003

Field	Value
Test Case ID	EMP-003
Title	Employee termination sets status and triggers access revocation
Module	Employee Management
Priority	P0

Field	Value
Severity	Critical
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Active employee emp-001 exists
- HR Manager is authenticated

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /employees/emp-001/terminate</code> with <code>{ "lastWorkingDate": "2025-06-30", "exitType": "voluntary", "reason": "Better opportunity" }</code>	HTTP 200
2	Inspect response body	Employee object with <code>status: "terminated"</code> , <code>last_working_date: "2025-06-30"</code>
3	Verify in <code>employment_history</code>	New row with <code>event_type = "exit"</code> , <code>effective_date = "2025-06-30"</code>
4	Verify in <code>users</code> table	Linked user account <code>is_active = FALSE</code>
5	Attempt login with terminated employee's credentials	HTTP 401 — account inactive
6	Verify in <code>audit_logs</code>	Entry with <code>action = "delete"</code> (soft), <code>resource = "employee"</code> , correct <code>old_values.status = "active"</code> , <code>new_values.status = "terminated"</code>

Expected Result: Employee terminated; history logged; user deactivated; audit trail complete.

Actual Result: _____

Pass / Fail: _____

EMP-004

Field	Value
Test Case ID	EMP-004
Title	Soft delete — terminated employee retained in database
Module	Employee Management
Priority	P0
Severity	Critical
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Employee emp-001 has been terminated (status = terminated)

Test Steps:

Step	Action	Expected Result
1	Send GET /employees (HR Manager token)	emp-001 NOT returned in active employee list
2	Send GET /employees?include_terminated=true	emp-001 IS returned with status: "terminated"
3	Query database directly: SELECT * FROM employees WHERE id = 'emp-001'	Row EXISTS; deleted_at = NULL; status = 'terminated'
4	Send GET /employees/emp-001 (HR Manager token)	HTTP 200 — terminated employee profile accessible
5	Verify employment_history for emp-001	All historical records intact
6	Verify payroll_entries for emp-001	All historical payroll records intact

Expected Result: Employee hidden from active lists but fully retrievable; all historical data preserved.

Actual Result: _____

Pass / Fail: _____

EMP-005

Field	Value
Test Case ID	EMP-005
Title	PII fields encrypted at rest — national ID not stored in plaintext
Module	Employee Management / Security
Priority	P0
Severity	Critical
Type	Security
Phase	V1
Status	Active

Preconditions:

- HR Manager authenticated
- Employee emp-001 exists

Test Steps:

Step	Action	Expected Result
1	Send PATCH /employees/emp-001 with { "nationalId": "1234567890123" }	HTTP 200
2	Query database: SELECT national_id FROM employees WHERE id = 'emp-001'	Value is NOT 1234567890123 — it is a Base64 AES-256-GCM ciphertext string
3	Send GET /employees/emp-001 with HR Manager token	nationalId field returned as decrypted plaintext "1234567890123"
4	Inspect audit_logs.new_values for the PATCH operation	nationalId field shows "[REDACTED]" — not the encrypted or plain value
5	Verify same for passportNumber and accountNumber on bank accounts	Both stored as ciphertext in DB; returned as plaintext via API; shown as [REDACTED] in audit log

Expected Result: PII encrypted in DB; decrypted via API; redacted in audit logs.

Actual Result: _____

Pass / Fail: _____

EMP-006

Field	Value
Test Case ID	EMP-006
Title	Bulk employee CSV import — valid rows imported, invalid rows reported
Module	Employee Management
Priority	P1
Severity	High
Type	Functional
Phase	V1
Status	Active

Preconditions:

- CSV file with 10 rows: 8 valid, 1 with missing `hireDate`, 1 with duplicate `employeeNumber`

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /employees/bulk-import</code> with the CSV file	HTTP 202 — <code>{ "jobId": "job-uuid", "status": "pending" }</code>
2	Poll <code>GET /employees/bulk-import/job-uuid</code> until <code>status = "done"</code>	Job completes within 30 seconds
3	Inspect job result	<code>{ "total": 10, "imported": 8, "failed": 2, "errors": [...] }</code>
4	Inspect <code>errors</code> array	Row 9: <code>{ "row": 9, "field": "hireDate", "message": "hireDate is required" }</code> ; Row 10: <code>{ "row": 10, "field": "employeeNumber", "message": "Duplicate employee number" }</code>
5	Verify in <code>employees</code> table	Exactly 8 new employees created with correct <code>company_id</code>
6	Verify invalid rows NOT imported	Rows 9 and 10 do not exist in DB

Expected Result: 8 valid rows imported; 2 error rows reported with field-level detail; partial success handled correctly.

Actual Result: _____

Pass / Fail: _____

Module 3 — Leave Management

LVE-001

Field	Value
Test Case ID	LVE-001
Title	Leave accrual — monthly accrual credits correct balance
Module	Leave Management

Field	Value
Priority	P0
Severity	Critical
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Leave type `Annual Leave` configured: `accrualType = "monthly"`, `accrualDays = 18` (1.5/month)
- Employee `emp-001` hired January 1, 2025; current date is March 31, 2025
- Leave balance for 2025 exists with `accrued_days = 0`

Test Steps:

Step	Action	Expected Result
1	Trigger monthly accrual job manually (or wait for scheduled run)	Job completes with status <code>done</code>
2	Send <code>GET /leave/balances/emp-001</code>	Response contains Annual Leave balance for 2025
3	Inspect <code>accrued_days</code> in response	<code>4.5</code> (3 months × 1.5 days/month)
4	Inspect <code>closing_days</code>	Equals <code>opening_days + 4.5 + adjusted_days - used_days</code>
5	Verify in <code>leave_balances</code> table	<code>accrued_days = 4.50</code> for <code>year = 2025</code>

Expected Result: 4.5 days accrued after 3 months; closing_days consistent with formula.

Actual Result: _____

Pass / Fail: _____

LVE-002

Field	Value
Test Case ID	LVE-002
Title	Leave request rejected when balance is insufficient
Module	Leave Management
Priority	P0
Severity	Critical
Type	Negative
Phase	V1
Status	Active

Preconditions:

- Employee emp-001 has closing_days = 2.0 for Annual Leave
- No pending or approved leave requests overlap the requested dates

Test Steps:

Step	Action	Expected Result
1	Send POST /leave/requests with { "leaveTypeId": "a1-id", "startDate": "2025-04-01", "endDate": "2025-04-05" } (5 working days)	HTTP 422
2	Inspect response body	{ "error": "INSUFFICIENT_BALANCE", "message": "Requested 5.0 days, available balance is 2.0 days" }
3	Verify in leave_requests table	No new row created
4	Verify in leave_balances table	used_days unchanged; closing_days unchanged

Expected Result: HTTP 422; no request created; balance unaffected.

Actual Result: _____

Pass / Fail: _____

LVE-003

Field	Value
Test Case ID	LVE-003
Title	Leave approval updates balance atomically
Module	Leave Management
Priority	P0
Severity	Critical
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Employee emp-001 has closing_days = 10.0 for Annual Leave
- Leave request lr-001 in pending status for 3 working days exists

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /leave/requests/lr-001/approve</code> with manager token	HTTP 200
2	Inspect response	<code>{ "status": "approved", "reviewedAt": "<timestamp>", "reviewedById": "<manager-id>" }</code>
3	Send <code>GET /leave/balances/emp-001</code>	<code>used_days</code> increased by <code>3.0</code> ; <code>closing_days</code> = <code>10.0 - 3.0 = 7.0</code>
4	Verify in <code>leave_requests</code> table	<code>status = "approved"</code> , <code>reviewed_by_id</code> and <code>reviewed_at</code> populated
5	Verify in <code>leave_balances</code> table	<code>used_days = 3.0</code> (or incremented by 3.0)
6	Verify in <code>audit_logs</code>	Entry with <code>action = "approve"</code> , <code>resource = "leave_request"</code> , <code>resource_id = "lr-001"</code>
7	Verify notification sent	Employee received in-app notification with type <code>leave.approved</code>

Expected Result: Balance decremented atomically; request approved; audit logged; employee notified.

Actual Result: _____

Pass / Fail: _____

LVE-004

Field	Value
Test Case ID	LVE-004
Title	Leave balance cannot go negative
Module	Leave Management
Priority	P0
Severity	Critical
Type	Boundary

Field	Value
Phase	V1
Status	Active

Preconditions:

- Employee has `closing_days = 0.0` for Annual Leave

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /leave/requests</code> for 1 day of Annual Leave	HTTP 422 <code>INSUFFICIENT_BALANCE</code>
2	Attempt to manually adjust balance via <code>PATCH /leave/balances/:id/adjust</code> with <code>adjustedDays: -5.0</code> when current <code>closing_days = 3.0</code>	HTTP 422 — adjustment would make balance negative
3	Attempt to approve a leave request via direct DB manipulation (bypass API) — simulate race condition by approving two overlapping requests simultaneously	Database transaction rollback; only one approval commits; balance remains non-negative
4	Verify in <code>leave_balances</code>	<code>closing_days >= 0</code> in all scenarios

Expected Result: No path (API or race condition) results in negative leave balance.

Actual Result: _____

Pass / Fail: _____

LVE-005

Field	Value
Test Case ID	LVE-005
Title	Holiday conflict blocks leave application on public holiday
Module	Leave Management
Priority	P1

Field	Value
Severity	High
Type	Negative
Phase	V1
Status	Active

Preconditions:

- Holiday calendar for employee's location has 2025-04-14 as a public holiday
- Employee has sufficient leave balance

Test Steps:

Step	Action	Expected Result
1	Send POST /leave/requests with startDate: "2025-04-14", endDate: "2025-04-14"	HTTP 422
2	Inspect response	{ "error": "HOLIDAY_CONFLICT", "message": "2025-04-14 is a public holiday" }
3	Send POST /leave/requests with startDate: "2025-04-12", endDate: "2025-04-16" (includes April 14)	HTTP 200 — request created with total_days = 4 (excludes the holiday)
4	Inspect total_days in created request	4.0 — not 5.0 (holiday excluded)

Expected Result: Single-day holiday application rejected; multi-day spanning a holiday succeeds with correct day count.

Actual Result: _____

Pass / Fail: _____

LVE-006

Field	Value
Test Case ID	LVE-006

Field	Value
Title	Year-end carry-forward respects policy cap
Module	Leave Management
Priority	P1
Severity	High
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Annual Leave policy: (accrualDays = 18), (carryForwardDays = 5)
- Employee has (closing_days = 12.0) at end of 2025 (unused balance)

Test Steps:

Step	Action	Expected Result
1	Trigger year-end rollover job for 2025 → 2026	Job completes
2	Send (GET /leave/balances/emp-001? year=2026)	2026 balance row exists
3	Inspect (opening_days) for 2026	(5.0) — capped at (carryForwardDays = 5), not (12.0)
4	Inspect 2025 balance after rollover	(closing_days) for 2025 finalized; lapsed days = (12.0 - 5.0 = 7.0)

Expected Result: Carry-forward capped at 5 days; excess lapsed; 2026 opening balance = 5.0.

Actual Result: _____

Pass / Fail: _____

Module 4 — Attendance Management

ATT-001

Field	Value
Test Case ID	ATT-001
Title	Clock-in and clock-out creates valid attendance record
Module	Attendance
Priority	P0
Severity	Critical
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Employee emp-001 is authenticated
- No existing attendance log for today

Test Steps:

Step	Action	Expected Result
1	Send POST /attendance/clock-in at 09:02 AM with IP and coordinates	HTTP 200; { "logDate": "<today>", "clockIn": "<timestamp>", "status": "present" }
2	Send POST /attendance/clock-out at 06:05 PM	HTTP 200; clockOut populated
3	Send GET /attendance/today	Returns today's record with both clockIn and clockOut
4	Verify in attendance_logs	work_minutes computed column = 541 (9h3m = 543 min, verify exact formula)
5	Verify source = "web"	Source field correctly recorded

Expected Result: Attendance record created; clock-in/out captured; work_minutes computed correctly.

Actual Result: _____

Pass / Fail: _____

ATT-002

Field	Value
Test Case ID	ATT-002
Title	Double clock-in on same day rejected
Module	Attendance
Priority	P0
Severity	High
Type	Negative
Phase	V1
Status	Active

Preconditions:

- Employee has already clocked in today (clock_in is set, clock_out is NULL)

Test Steps:

Step	Action	Expected Result
1	Send POST /attendance/clock-in again	HTTP 409
2	Inspect response body	{ "error": "ALREADY_CLOCKED_IN", "message": "You have already clocked in today" }
3	Verify attendance_logs	Still only one record for today; clock_in unchanged

Expected Result: HTTP 409; existing record unchanged; no duplicate created.

Actual Result: _____

Pass / Fail: _____

ATT-003

Field	Value
Test Case ID	ATT-003
Title	HR manual attendance correction is audit logged
Module	Attendance
Priority	P0
Severity	High
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Attendance log (att-001) for employee (emp-001) has (clock_in = NULL) (missed punch)
- HR Manager is authenticated

Test Steps:

Step	Action	Expected Result
1	Send <code>PATCH /attendance/att-001/correct</code> with <code>{ "clockIn": "2025-04-10T09:00:00Z", "notes": "Employee reported system issue" }</code>	HTTP 200
2	Inspect response	<code>clockIn</code> updated; <code>source = "manual"</code>
3	Verify in <code>audit_logs</code>	Entry with <code>action = "update"</code> , <code>resource = "attendance_log"</code> , <code>old_values.clock_in = null</code> , <code>new_values.clock_in = "2025-04-10T09:00:00Z"</code> , <code>new_values.source = "manual"</code>
4	Verify <code>is_approved = false</code> in DB	Correction requires separate approval step
5	Send <code>POST /attendance/att-001/approve</code> with manager token	HTTP 200; <code>is_approved = true</code>

Expected Result: Manual correction logged with audit trail; source flagged as manual; requires approval.

Actual Result: _____

Pass / Fail: _____

Module 5 — Payroll

PAY-001

Field	Value
Test Case ID	PAY-001
Title	Payroll computation — net pay equals gross minus total deductions
Module	Payroll
Priority	P0
Severity	Critical
Type	Functional

Field	Value
Phase	V1
Status	Active

Preconditions:

- Employee assigned salary structure: BASIC = 50,000; HRA = 20,000; TDS = 7,000; PF = 6,000
- No LOP days; full month (22 working days worked)
- Payroll cycle `cycle-jan-2025` in `draft` status

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /payroll/cycles/cycle-jan-2025/run</code>	HTTP 202 — async job queued
2	Poll <code>GET /payroll/cycles/cycle-jan-2025/status</code> until <code>status = "processing_done"</code>	Completes within 30 seconds
3	Send <code>GET /payroll/cycles/cycle-jan-2025/entries/emp-001</code>	Entry returned
4	Verify <code>gross_pay = 70,000</code>	Sum of BASIC(50k) + HRA(20k) = 70,000
5	Verify <code>total_deductions = 13,000</code>	Sum of TDS(7k) + PF(6k) = 13,000
6	Verify <code>net_pay = 57,000</code>	<code>gross_pay - total_deductions = 57,000</code>
7	Verify <code>lop_days = 0</code> , <code>paid_days = 22</code>	No LOP applied
8	Send <code>GET /payroll/cycles/cycle-jan-2025/entries/emp-001/components</code>	All 4 components listed with correct amounts

Expected Result: net_pay = gross - deductions; all component amounts correct; formula: 70,000 - 13,000 = 57,000.

Actual Result: _____

Pass / Fail: _____

PAY-002

Field	Value
Test Case ID	PAY-002
Title	Loss of Pay (LOP) correctly reduces net pay
Module	Payroll
Priority	P0
Severity	Critical
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Employee salary: BASIC = 60,000/month; full salary = 60,000
- Period has 22 working days; employee has `lop_days = 2` (absent without approved leave)
- Payroll cycle in draft status

Test Steps:

Step	Action	Expected Result
1	Run payroll cycle	Completes
2	Send <code>GET /payroll/cycles/:id/entries/emp-001</code>	Entry returned
3	Verify <code>lop_days = 2.0</code>	Correct LOP recorded
4	Verify <code>paid_days = 20.0</code>	<code>22 - 2 = 20</code>
5	Verify <code>gross_pay = 54,545.45</code>	<code>60,000 / 22 × 20 = 54,545.45</code> (rounded to 2 decimal places)
6	Verify component BASIC reflects proration	<code>BASIC = 45,454.55</code> (50,000/22×20)

Expected Result: LOP reduces paid_days and gross_pay proportionally; individual component amounts prorated.

Actual Result: _____

Pass / Fail: _____

PAY-003

Field	Value
Test Case ID	PAY-003
Title	Payroll cycle state machine — approved cycle cannot be re-run
Module	Payroll
Priority	P0
Severity	High
Type	Negative
Phase	V1
Status	Active

Preconditions:

- Payroll cycle cycle-jan-2025 is in approved status

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /payroll/cycles/cycle-jan-2025/run</code>	HTTP 422
2	Inspect response	<code>{ "error": "INVALID_CYCLE_STATE", "message": "Cannot run a cycle in 'approved' status" }</code>
3	Send <code>POST /payroll/cycles/cycle-jan-2025/approve</code>	HTTP 422 — already approved
4	Send <code>POST /payroll/cycles/cycle-jan-2025/disburse</code>	HTTP 200 — disburse is valid from approved
5	After disbursement, send <code>POST /payroll/cycles/cycle-jan-2025/run</code>	HTTP 422 — disbursed cycle cannot be re-run

Expected Result: Only valid state transitions succeed; invalid transitions return 422 with clear error.

Actual Result: _____

Pass / Fail: _____

PAY-004

Field	Value
Test Case ID	PAY-004
Title	Payslip PDF generated and accessible only to authorized users
Module	Payroll
Priority	P0
Severity	Critical
Type	Security
Phase	V1
Status	Active

Preconditions:

- Payroll cycle `cycle-jan-2025` in `approved` status
- `POST /payroll/cycles/cycle-jan-2025/payslips/generate` has been triggered

Test Steps:

Step	Action	Expected Result
1	Poll until payslip generation job completes	Payslip records exist in <code>payslips</code> table
2	Send <code>GET /payslips</code> with <code>emp-001</code> token	Returns list of payslips for emp-001 only
3	Send <code>GET /payslips/:id</code> with <code>emp-001</code> token	Returns <code>{ "fileUrl": "<signed-s3-url>", "generatedAt": "..." }</code>
4	Access the signed S3 URL	PDF downloads successfully; contains correct employee name, period, net pay
5	Send <code>GET /payslips/:id</code> for emp-002's payslip using emp-001's token	HTTP 403 or 404 — cannot access another employee's payslip
6	After 60 minutes, attempt to access the original signed URL again	HTTP 403 from S3 — signed URL has expired

Expected Result: Employee can only access their own payslips; signed URLs expire after 60 minutes.

Actual Result: _____

Pass / Fail: _____

PAY-005

Field	Value
Test Case ID	PAY-005
Title	Bank transfer file contains correct data for all disbursed employees
Module	Payroll
Priority	P1
Severity	High
Type	Functional

Field	Value
Phase	V1
Status	Active

Preconditions:

- Payroll cycle `cycle-jan-2025` in `disbursed` status
- 5 employees with different bank accounts and net pay values

Test Steps:

Step	Action	Expected Result
1	Send <code>GET /payroll/cycles/cycle-jan-2025/bank-file</code>	HTTP 200; file downloaded
2	Verify file format	CSV with headers: <code>employee_number</code> , <code>employee_name</code> , <code>bank_code</code> , <code>account_number</code> , <code>amount</code>
3	Verify employee count in file	Exactly 5 rows (one per employee)
4	Verify amounts	Each row's <code>amount</code> matches the corresponding <code>payroll_entries.net_pay</code>
5	Verify account numbers	Each row's <code>account_number</code> matches the last 4 digits visible; full number decrypted correctly
6	Verify <code>status = "disbursed"</code> for all entries in DB	All 5 entries have <code>status = "disbursed"</code>

Expected Result: Bank file contains all 5 employees; amounts match payroll entries; account details correct.

Actual Result: _____

Pass / Fail: _____

Module 6 — Documents & Compliance

DOC-001

Field	Value
Test Case ID	DOC-001
Title	Policy acknowledgement tracked per employee per version
Module	Documents & Compliance
Priority	P0
Severity	High
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Policy (POSH Policy v1.0) is published with (is_mandatory = TRUE)
- 3 employees exist in the company

Test Steps:

Step	Action	Expected Result
1	Employee 1 sends <code>POST /compliance/policies/:id/acknowledge</code>	HTTP 200; acknowledgement record created
2	Employee 1 sends <code>POST /compliance/policies/:id/acknowledge</code> again	HTTP 409 — already acknowledged (idempotent guard)
3	HR sends <code>GET /compliance/policies/:id/acknowledgements</code>	Returns: Employee 1 acknowledged; Employees 2 and 3 pending
4	HR publishes <code>POSH Policy v2.0</code> (new version of same policy)	New policy record created with <code>version = "2.0"</code>
5	HR sends <code>GET /compliance/policies/:new-id/acknowledgements</code>	All 3 employees show as pending for v2.0 (v1.0 acknowledgement does not carry over)
6	Verify <code>audit_logs</code>	Acknowledgement creation logged with <code>user_id = emp-1</code> , <code>resource = "policy_acknowledgement"</code>

Expected Result: Acknowledgements are version-specific; v1 ack does not count for v2; idempotent.

Actual Result: _____

Pass / Fail: _____

DOC-002

Field	Value
Test Case ID	DOC-002
Title	eSign — signed document hash validates tampering detection
Module	Documents & Compliance
Priority	P0
Severity	Critical
Type	Security

Field	Value
Phase	V1
Status	Active

Preconditions:

- eSign request `es-001` exists in `pending` status

Test Steps:

Step	Action	Expected Result
1	Employee signs document: <code>POST</code> <code>/compliance/esign/es-001/sign</code>	HTTP 200; <code>status = "signed"</code> , <code>file_hash</code> populated in DB
2	Retrieve signed document: <code>GET</code> <code>/compliance/esign/es-001/download</code>	PDF downloaded
3	Compute SHA-256 of downloaded PDF	Hash matches <code>esign_requests.file_hash</code> in DB
4	Simulate tampering: modify one byte of the signed PDF file in S3	—
5	Re-download and compute SHA-256	Hash does NOT match stored <code>file_hash</code> — tampering detected
6	Attempt to sign <code>es-001</code> again	HTTP 409 — <code>{ "error": "ALREADY_SIGNED" }</code>
7	Attempt to sign after <code>expires_at</code> has passed	HTTP 422 — <code>{ "error": "REQUEST_EXPIRED" }</code>

Expected Result: Hash detects post-signature tampering; re-signing and expired request attempts blocked.

Actual Result: _____

Pass / Fail: _____

Module 7 — Audit Log

AUD-001

Field	Value
Test Case ID	AUD-001
Title	All mutating operations generate audit log entries
Module	Audit Log
Priority	P0
Severity	Critical
Type	Functional
Phase	V1
Status	Active

Preconditions:

- HR Manager authenticated
- Clean audit log (or note initial count)

Test Steps:

Step	Action	Expected Result
1	<code>POST /employees</code> (create employee)	Audit entry: <code>action = "create"</code> , <code>resource = "employee"</code>
2	<code>PATCH /employees/:id</code> (update name)	Audit entry: <code>action = "update"</code> , <code>old_values.firstName</code> = old name, <code>new_values.firstName</code> = new name
3	<code>DELETE /employees/:id</code> (soft delete)	Audit entry: <code>action = "delete"</code> , <code>resource = "employee"</code>
4	<code>GET /employees</code> (read operation)	NO audit entry created (reads not audited)
5	<code>POST /leave/requests/:id/approve</code> (approve action)	Audit entry: <code>action = "approve"</code> , <code>resource = "leave_request"</code>
6	<code>POST /payroll/cycles/:id/run</code>	Audit entry: <code>action = "create"</code> , <code>resource = "payroll_cycle"</code> job dispatch
7	Verify all entries	All contain <code>company_id</code> , <code>user_id</code> , <code>ip_address</code> , <code>created_at</code> ; none contain plaintext PII

Expected Result: Every mutation audited; reads not audited; all required fields present; PII excluded.

Actual Result: _____

Pass / Fail: _____

AUD-002

Field	Value
Test Case ID	AUD-002
Title	Audit log is append-only — entries cannot be modified or deleted via API
Module	Audit Log
Priority	P0
Severity	Critical

Field	Value
Type	Security
Phase	V1
Status	Active

Preconditions:

- At least one audit log entry exists with `id = 1001`
- Admin user is authenticated

Test Steps:

Step	Action	Expected Result
1	Attempt <code>DELETE /compliance/audit-logs/1001</code>	HTTP 405 — Method Not Allowed (no delete endpoint exists)
2	Attempt <code>PATCH /compliance/audit-logs/1001</code>	HTTP 405 — Method Not Allowed (no update endpoint exists)
3	Attempt direct SQL: <code>UPDATE audit_logs SET action = 'fake'</code>	PostgreSQL permission denied — app DB user has no UPDATE on audit_logs
4	Attempt direct SQL: <code>DELETE FROM audit_logs WHERE id = 1001</code>	PostgreSQL permission denied — app DB user has no DELETE on audit_logs
5	<code>GET /compliance/audit-logs?resource=employee</code>	Returns read-only entries correctly

Expected Result: Audit log has no update or delete endpoints; DB user lacks write permission on audit_logs.

Actual Result: _____

Pass / Fail: _____

AUD-003

Field	Value
Test Case ID	AUD-003
Title	Audit log does not block HTTP response — fire-and-forget

Field	Value
Module	Audit Log
Priority	P1
Severity	High
Type	Performance
Phase	V1
Status	Active

Preconditions:

- Audit service is configured to write asynchronously

Test Steps:

Step	Action	Expected Result
1	Simulate audit DB write latency of 500ms (using mock/test flag)	—
2	Send <code>POST /employees</code> and measure response time	HTTP 201 returned in < 150ms (does not wait for audit write)
3	After 500ms, query <code>audit_logs</code>	Entry exists — write completed asynchronously
4	Simulate audit DB write failure	HTTP 201 still returned; audit failure logged to app logger (<code>ERROR level</code>); no exception propagated to client

Expected Result: Audit logging never delays HTTP response; audit failure does not break the business operation.

Actual Result: _____

Pass / Fail: _____

Module 8 — Recruitment & ATS `[V2]`

REC-001

Field	Value
Test Case ID	REC-001

Field	Value
Title	Application pipeline stage transition — only valid moves succeed
Module	Recruitment
Priority	P0
Severity	Critical
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Job requisition `req-001` in `open` status
- Candidate application `app-001` in `applied` stage

Test Steps:

Step	Action	Expected Result
1	<code>PATCH /recruitment/applications/app-001/stage</code> with <code>{ "stage": "screening" }</code>	HTTP 200; stage = <code>screening</code>
2	<code>PATCH /recruitment/applications/app-001/stage</code> with <code>{ "stage": "interview" }</code>	HTTP 200; stage = <code>interview</code>
3	<code>PATCH /recruitment/applications/app-001/stage</code> with <code>{ "stage": "applied" }</code> (backwards)	HTTP 422 — <code>{ "error": "INVALID_STAGE_TRANSITION", "message": "Cannot move from 'interview' to 'applied'" }</code>
4	<code>POST /recruitment/applications/app-001/reject</code> with <code>{ "reason": "skills_mismatch" }</code>	HTTP 200; stage = <code>rejected</code>
5	<code>PATCH /recruitment/applications/app-001/stage</code> with <code>{ "stage": "interview" }</code> (from rejected)	HTTP 422 — <code>{ "error": "TERMINAL_STAGE", "message": "Application is in a terminal stage" }</code>
6	Verify in <code>audit_logs</code>	Each stage change logged with <code>old_values.stage</code> and <code>new_values.stage</code>

Expected Result: Forward transitions succeed; backward transitions blocked; terminal stages locked.

Actual Result: _____

Pass / Fail: _____

REC-002

Field	Value
Test Case ID	REC-002
Title	Accepted offer triggers automatic employee creation

Field	Value
Module	Recruitment
Priority	P0
Severity	Critical
Type	Integration
Phase	V2
Status	Active

Preconditions:

- Offer `offer-001` in `sent` status for candidate `cand-001`
- Offer CTC = 72,000; join date = 2025-07-01

Test Steps:

Step	Action	Expected Result
1	<code>POST</code> <code>/recruitment/offers/offer-001/accept</code>	HTTP 200
2	Verify offer status	<code>status = "accepted"</code> , <code>responded_at</code> populated
3	Verify in <code>employees</code> table	New employee record created with <code>email = cand-001.email</code> , <code>hire_date = "2025-07-01"</code> , <code>status = "active"</code>
4	Verify in <code>employee_salaries</code>	Salary record with <code>ctc_annual = 72,000</code> , <code>effective_from = "2025-07-01"</code>
5	Verify application stage	<code>app-001</code> stage = <code>hired</code>
6	Verify in <code>employee_onboarding</code>	Onboarding instance created if default template exists
7	Verify welcome email queued	<code>background_jobs</code> contains a <code>send_welcome</code> job for the new employee

Expected Result: Offer acceptance atomically creates employee, salary record, and triggers onboarding.

Actual Result: _____

Pass / Fail: _____

Module 9 — Performance Management [V2]

PERF-001

Field	Value
Test Case ID	PERF-001
Title	OKR alignment — child goal progress rolls up to parent
Module	Performance Management
Priority	P1
Severity	High
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Company goal [goal-co-001]: title "Reach 1000 customers", [target_value = 1000]
- Individual goal [goal-ind-001] (child of [goal-co-001]): title "Close 50 enterprise accounts", [target_value = 50], [current_value = 0]

Test Steps:

Step	Action	Expected Result
1	<code>POST /goals/goal-ind-001/check-in</code> with <code>{ "progressValue": 20, "notes": "Q1 done" }</code>	HTTP 200
2	<code>GET /goals/goal-ind-001</code>	<code>current_value = 20</code> ; progress = <code>20/50 = 40%</code>
3	<code>GET /goals/tree</code>	Tree shows <code>goal-co-001</code> with child <code>goal-ind-001</code> ; child shows 40% progress
4	<code>POST /goals/goal-ind-001/check-in</code> with <code>{ "progressValue": 50, "status": "completed" }</code>	HTTP 200
5	<code>GET /goals/goal-ind-001</code>	<code>status = "completed"</code> , <code>current_value = 50</code>
6	Verify <code>goal_check_ins</code> table	2 rows for <code>goal-ind-001</code> with timestamps and notes

Expected Result: Check-ins update `current_value`; status transitions to completed; full audit trail in check-ins.

Actual Result: _____

Pass / Fail: _____

PERF-002

Field	Value
Test Case ID	PERF-002
Title	360° review — peer feedback anonymous; manager sees aggregated rating
Module	Performance Management
Priority	P1
Severity	High
Type	Functional

Field	Value
Phase	V2
Status	Active

Preconditions:

- Review cycle (cycle-2025) active
- Employee (emp-001) has self, manager, 2 peer review forms assigned

Test Steps:

Step	Action	Expected Result
1	Peer 1 submits feedback for (emp-001) with (overall_rating = 4.0)	HTTP 200; (reviewer_type = "peer")
2	Peer 2 submits feedback for (emp-001) with (overall_rating = 3.5)	HTTP 200
3	Manager views peer feedback for (emp-001)	Sees peer ratings aggregated; individual peer identities NOT revealed if anonymity enabled
4	(emp-001) sends (GET /performance/reviews)	Cannot see individual peer identities; can see their own self-review
5	HR views calibration: (GET /performance/reviews/calibration)	Sees all reviewers including peer identities (HR exempt from anonymity)
6	HR calibrates: (PATCH /performance/reviews/:id/calibrate) with ({ "finalRating": 3.8 })	HTTP 200; (final_rating = 3.8) differs from (overall_rating)
7	Verify (audit_logs)	Calibration logged with (old_values.final_rating = null), (new_values.final_rating = 3.8)

Expected Result: Peer anonymity enforced for employee and manager views; HR sees all; calibration override audited.

Actual Result: _____

Pass / Fail: _____

Module 10 — Analytics & Reporting

RPT-001

Field	Value
Test Case ID	RPT-001
Title	Headcount report returns accurate counts by department
Module	Reporting
Priority	P1
Severity	High
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Company has: Engineering dept (12 active, 2 terminated), HR dept (4 active), Product dept (6 active)

Test Steps:

Step	Action	Expected Result
1	<code>GET /reports/headcount?groupBy=department</code>	HTTP 200; JSON array
2	Verify Engineering count	<code>12</code> — terminated employees excluded by default
3	<code>GET /reports/headcount?groupBy=department&include_terminated=true</code>	Engineering count = <code>14</code>
4	<code>GET /reports/headcount?groupBy=department&date=2025-01-01</code>	Returns headcount as it was on Jan 1 (point-in-time)
5	Verify query uses read replica	Trace logs show query routed to <code>READ_REPLICA</code> connection pool

Expected Result: Counts accurate; terminated excluded by default; point-in-time query works; read replica used.

Actual Result: _____

Pass / Fail: _____

RPT-002

Field	Value
Test Case ID	RPT-002
Title	Report export generates valid XLSX file asynchronously
Module	Reporting
Priority	P1
Severity	Medium
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Saved report rpt-001 configured with headcount by department filters

Test Steps:

Step	Action	Expected Result
1	<code>GET /reports/saved/rpt-001/export?format=xlsx</code>	HTTP 202; <code>{ "jobId": "export-job-1", "status": "pending" }</code>
2	Poll <code>GET /reports/export/export-job-1</code>	Status transitions: <code>pending → running → done</code>
3	When <code>status = "done"</code> , inspect <code>downloadUrl</code>	Valid signed S3 URL returned
4	Download the XLSX file	HTTP 200; file downloads; MIME type <code>application/vnd.openxmlformats-officedocument.spreadsheetml.sheet</code>
5	Open file in Excel / verify programmatically	Contains correct headers; data matches API response; no formatting errors
6	Verify notification sent	HR user received in-app notification "Your report is ready for download"

Expected Result: XLSX generated asynchronously; downloadable via signed URL; user notified on completion.

Actual Result: _____

Pass / Fail: _____

Module 11 — Webhooks & Integrations [V2]

WH-001

Field	Value
Test Case ID	WH-001
Title	Webhook delivery includes valid HMAC-SHA256 signature
Module	Webhooks
Priority	P1
Severity	High
Type	Security

Field	Value
Phase	V2
Status	Active

Preconditions:

- Webhook registered: URL = `https://webhook.site/test`, secret = `mysecret123`, events = `["employee.created"]`
- Webhook receiver server is running and accessible

Test Steps:

Step	Action	Expected Result
1	Create a new employee (triggers <code>employee.created</code> event)	Employee created; event emitted to webhook queue
2	Inspect webhook delivery at receiver	HTTP POST received within 30 seconds
3	Inspect <code>X-HR-Signature-256</code> header	Present; value = <code>sha256=<hmac></code>
4	Verify signature: compute <code>HMAC-SHA256(mysecret123, raw_body)</code>	Computed signature matches header value
5	Modify the payload body by one character and re-verify signature	Signature does NOT match — integrity protection verified
6	Inspect payload body	Contains <code>{ "event": "employee.created", "companyId": "...", "resourceId": "...", "timestamp": "..." }</code> — no PII fields
7	Verify in <code>webhook_deliveries</code> table	Row with <code>is_success = true</code> , <code>response_status = 200</code> , <code>duration_ms</code> recorded

Expected Result: HMAC signature valid; payload contains no PII; delivery logged.

Actual Result: _____

Pass / Fail: _____

WH-002

Field	Value
Test Case ID	WH-002
Title	Webhook auto-disabled after 10 consecutive failures
Module	Webhooks
Priority	P1
Severity	High
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Webhook registered with a URL that returns HTTP 500

Test Steps:

Step	Action	Expected Result
1	Trigger 10 webhook-emitting events (e.g., create 10 employees)	10 delivery attempts made; all fail with <code>is_success = false</code>
2	Inspect <code>webhooks.failure_count</code> after 10 failures	<code>failure_count = 10</code>
3	Inspect <code>webhooks.is_active</code>	<code>is_active = FALSE</code> — webhook auto-disabled
4	Trigger another employee creation	No delivery attempt made (webhook inactive)
5	Verify HR admin notification	Alert sent: "Webhook <name> has been disabled after 10 consecutive failures"
6	HR re-activates webhook via <code>PATCH /integrations/webhooks/:id</code> with <code>{ "isActive": true }</code>	Webhook re-enabled; <code>failure_count</code> reset to 0

Expected Result: Webhook disabled at 10 failures; admin notified; can be manually re-enabled.

Actual Result: _____

Pass / Fail: _____

Module 12 — Multi-tenancy & Data Isolation

MT-001

Field	Value
Test Case ID	MT-001
Title	All API list endpoints return only data scoped to authenticated company
Module	Multi-tenancy
Priority	P0
Severity	Critical
Type	Security
Phase	V1
Status	Active

Preconditions:

- Company A has 15 employees, 3 departments, 2 payroll cycles
- Company B has 8 employees, 2 departments, 1 payroll cycle
- Company A user is authenticated

Test Steps:

Step	Action	Expected Result
1	<code>GET /employees</code> with Company A token	Returns exactly 15 employees; no Company B employees present
2	<code>GET /departments</code> with Company A token	Returns 3 departments; no Company B departments
3	<code>GET /payroll/cycles</code> with Company A token	Returns 2 cycles; Company B's cycle not present
4	<code>GET /leave/types</code> with Company A token	Returns only Company A leave types
5	<code>GET /compliance/audit-logs</code> with Company A token	Returns only Company A audit entries
6	Repeat steps 1-5 with Company B token	Company B sees only their own data; Company A data not visible

Expected Result: Perfect data isolation; every list endpoint scoped to authenticated company.

Actual Result: _____

Pass / Fail: _____

MT-002

Field	Value
Test Case ID	MT-002
Title	Prisma tenant scope extension applied to all query types
Module	Multi-tenancy
Priority	P0
Severity	Critical
Type	Security
Phase	V1
Status	Active

Preconditions:

- Company A token; Company B employee ID `emp-B-1` is known

Test Steps:

Step	Action	Expected Result
1	<code>GET /employees/emp-B-1</code> using Company A token	HTTP 404 — not exposed even to confirm existence
2	<code>PATCH /employees/emp-B-1</code> with <code>{ "firstName": "Hacked" }</code> using Company A token	HTTP 404 — no update applied
3	<code>DELETE /employees/emp-B-1</code> using Company A token	HTTP 404
4	Verify in DB after steps 1-3	<code>emp-B-1</code> record unchanged in DB; <code>deleted_at</code> still NULL
5	Verify in <code>audit_logs</code> for Company A	Steps 2 and 3 logged as <code>attempt_failed</code> with <code>resource_id = emp-B-1</code> ; Company A cannot see audit for Company B

Expected Result: Cross-tenant modification attempts silently return 404; no data changed; failed attempts logged in requesting company's audit log.

Actual Result: _____

Pass / Fail: _____

Module 13 — Performance & Boundary Tests

PERF-LOAD-001

Field	Value
Test Case ID	PERF-LOAD-001
Title	<code>GET /employees</code> p95 response time under 120ms with 1,000 employees
Module	Performance
Priority	P1
Severity	High

Field	Value
Type	Performance
Phase	V1
Status	Active

Preconditions:

- Staging environment seeded with 1,000 active employees for one company
- All database indexes created and verified
- k6 load test script prepared

Test Steps:

Step	Action	Expected Result
1	Run k6 script: 50 virtual users, <code>GET /employees?page=1&per_page=25</code> , 60 seconds duration	k6 test executes
2	Inspect p50 latency	< 50ms
3	Inspect p95 latency	< 120ms
4	Inspect p99 latency	< 300ms
5	Inspect error rate	0% — no HTTP 5xx or 4xx (except expected 401 for unauthenticated)
6	Verify query plan: <code>EXPLAIN ANALYZE</code> on the underlying query	Index scan on <code>(company_id)</code> used; no sequential scan

Expected Result: p95 < 120ms; p99 < 300ms; 0% errors; index used.

Actual Result: _____

Pass / Fail: _____

PERF-LOAD-002

Field	Value
Test Case ID	PERF-LOAD-002
Title	Payroll compute for 500 employees completes within 30 seconds

Field	Value
Module	Payroll / Performance
Priority	P1
Severity	High
Type	Performance
Phase	V1
Status	Active

Preconditions:

- 500 active employees with assigned salary structures and attendance logs for the period
- Payroll cycle `cycle-perf-test` in `draft` status

Test Steps:

Step	Action	Expected Result
1	Record timestamp T1; send <code>POST /payroll/cycles/cycle-perf-test/run</code>	HTTP 202; job queued
2	Poll <code>GET /payroll/cycles/cycle-perf-test/status</code> until <code>status = "processing_done"</code>	Status updates
3	Record timestamp T2 when status = done	<code>T2 - T1 < 30 seconds</code>
4	Verify entry count	500 <code>payroll_entries</code> rows created
5	Spot-check 10 random entries	<code>net_pay = gross_pay - total_deductions</code> for all 10
6	Verify no duplicate entries	<code>SELECT employee_id, COUNT(*) FROM payroll_entries WHERE cycle_id = '...' GROUP BY employee_id HAVING COUNT(*) > 1</code> returns 0 rows

Expected Result: 500 employees processed in < 30s; all entries correct; no duplicates.

Actual Result: _____

Pass / Fail: _____

BOUND-001

Field	Value
Test Case ID	BOUND-001
Title	Boundary — employee with zero salary processes without error
Module	Payroll / Boundary
Priority	P2
Severity	Medium
Type	Boundary
Phase	V1
Status	Active

Preconditions:

- Employee (emp-intern) is an unpaid intern with (ctc_annual = 0)

Test Steps:

Step	Action	Expected Result
1	Run payroll cycle including (emp-intern)	Completes without error
2	(GET /payroll/cycles/:id/entries/emp-intern)	(gross_pay = 0), (net_pay = 0), (total_deductions = 0)
3	Attempt to generate payslip for (emp-intern)	Payslip generated (PDF with zero amounts, not an error)
4	Verify bank transfer file	(emp-intern) row present with (amount = 0.00)

Expected Result: Zero-salary employee processed gracefully; no division-by-zero errors.

Actual Result: _____

Pass / Fail: _____

BOUND-002

Field	Value
Test Case ID	BOUND-002
Title	Boundary — employee hired on last day of payroll period
Module	Payroll / Boundary
Priority	P2
Severity	Medium
Type	Boundary
Phase	V1
Status	Active

Preconditions:

- Payroll period: 2025-04-01 to 2025-04-30 (22 working days)
- Employee `emp-last-day` hired on 2025-04-30 (last day of period)

Test Steps:

Step	Action	Expected Result
1	Run payroll for April 2025	Completes without error
2	<code>GET /payroll/cycles/:id/entries/emp-last-day</code>	Entry exists
3	Verify <code>working_days = 22</code> , <code>paid_days = 1</code>	One working day in the period
4	Verify <code>gross_pay = salary / 22 × 1</code>	Prorated to 1/22nd of monthly salary
5	Verify <code>lop_days = 0</code>	Not absent on their one day

Expected Result: Single-day employee prorated correctly; no errors on boundary hire date.

Postconditions:

- Payroll entry for `emp-last-day` persisted in `payroll_entries` with correct proration values
- No other employees in the cycle are affected by the boundary hire

Test Data:

- `emp-last-day`: `ctc_annual = 60,000`; monthly salary = `5,000`
- Expected `gross_pay = 5,000 / 22 × 1 = 227.27`

Actual Result: _____

Pass / Fail: _____

Notes: Also verify the inverse case — employee whose `last_working_date` falls on the first day of the period is paid for exactly 1 day and not included in subsequent cycles.

BOUND-003

Field	Value
Test Case ID	BOUND-003
Title	Boundary — leave request spanning a weekend counts only working days
Module	Leave Management / Boundary
Priority	P2
Severity	Medium
Type	Boundary
Phase	V1
Status	Active

Preconditions:

- Employee `emp-001` has `closing_days = 10.0` for Annual Leave
- Work schedule is `weekdays` (Monday-Friday)
- Date range chosen: Friday 2025-04-11 to Monday 2025-04-14 (spans weekend)

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /leave/requests</code> with <code>startDate: "2025-04-11"</code> , <code>endDate: "2025-04-14"</code>	HTTP 200 — request created
2	Inspect <code>total_days</code> in response	<code>2.0</code> — Friday and Monday only; Saturday and Sunday excluded
3	After approval, inspect <code>leave_balances.used_days</code>	Increased by <code>2.0</code> , not <code>4.0</code>
4	Send same request for employee with <code>work_schedule = "shifts"</code>	<code>total_days = 4.0</code> — all calendar days counted for shift workers
5	Send <code>POST /leave/requests</code> with <code>startDate: "2025-04-12"</code> , <code>endDate: "2025-04-13"</code> (Saturday-Sunday only)	HTTP 422 — <code>{ "error": "NO_WORKING_DAYS", "message": "Selected dates contain no working days" }</code>

Expected Result: Weekend days excluded for weekday schedule; all days counted for shift schedule; weekend-only request rejected.

Postconditions:

- `leave_balances.used_days` decremented by exactly 2.0 for weekday employee
- No leave record created for weekend-only request

Test Data:

- Employee work_schedule: `weekdays`
- Leave type: Annual Leave

Actual Result: _____

Pass / Fail: _____

BOUND-004

Field	Value
Test Case ID	BOUND-004
Title	Boundary — maximum VARCHAR field lengths enforced

Field	Value
Module	Employee Management / Boundary
Priority	P2
Severity	Low
Type	Boundary
Phase	V1
Status	Active

Preconditions:

- HR Manager authenticated

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /employees</code> with <code>firstName</code> = string of exactly 100 characters	HTTP 201 — maximum length accepted
2	Send <code>POST /employees</code> with <code>firstName</code> = string of 101 characters	HTTP 422 — <code>{ "error": "VALIDATION_ERROR", "field": "firstName", "message": "Must be 100 characters or fewer" }</code>
3	Send <code>POST /employees</code> with <code>email</code> = string of exactly 255 characters (valid email format)	HTTP 201 — maximum email length accepted
4	Send <code>POST /employees</code> with <code>email</code> = 256-character string	HTTP 422 — validation error on <code>email</code> field
5	Send <code>POST /employees</code> with <code>employeeNumber</code> = exactly 50 characters	HTTP 201 — accepted
6	Send <code>POST /employees</code> with <code>employeeNumber</code> = 51 characters	HTTP 422 — validation error
7	Send <code>POST /employees</code> with empty string <code>""</code> for <code>firstName</code>	HTTP 422 — <code>{ "field": "firstName", "message": "firstName is required" }</code>

Expected Result: Max length accepted; max+1 rejected with field-level error; empty string rejected.

Postconditions:

- No employees created from steps 2, 4, 6, 7
- Employees from steps 1, 3, 5 created successfully

Test Data:

- 100-char string: "A".repeat(100)
- 101-char string: "A".repeat(101)

Actual Result: _____

Pass / Fail: _____

Module 14 — Learning & Development [V2]

LMS-001

Field	Value
Test Case ID	LMS-001
Title	Course enrollment progress tracked correctly from 0% to completion
Module	Learning & Development
Priority	P1
Severity	High
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Course [course-001] exists: [passing_score = 70], [is_mandatory = false]
- Employee [emp-001] is authenticated

Test Steps:

Step	Action	Expected Result
1	POST /lms/courses/course-001/enroll	HTTP 200; enrollment created with status = "enrolled", progress_pct = 0
2	PATCH /lms/courses/course-001/progress with { "progressPct": 50 }	HTTP 200; status = "in_progress", progress_pct = 50
3	PATCH /lms/courses/course-001/progress with { "progressPct": 100 }	HTTP 200; progress_pct = 100
4	POST /lms/courses/course-001/complete with { "score": 85 }	HTTP 200; status = "completed", score = 85, completed_at populated
5	Inspect certificate_url in response	S3 signed URL to generated certificate PDF
6	POST /lms/courses/course-001/complete again (idempotency check)	HTTP 409 — { "error": "ALREADY_COMPLETED" }
7	Verify in course_enrollments table	Single row; status = "completed", score = 85.00

Expected Result: Progress tracked correctly 0→50→100%; completion generates certificate; re-completion blocked.

Postconditions:

- course_enrollments row with status = "completed" and score = 85.00
- Certificate PDF accessible via signed URL

Actual Result: _____

Pass / Fail: _____

LMS-002

Field	Value
Test Case ID	LMS-002
Title	Mandatory course assignment — employee blocked from marking complete below passing score

Field	Value
Module	Learning & Development
Priority	P1
Severity	High
Type	Negative
Phase	V2
Status	Active

Preconditions:

- Course `course-compliance` exists with `passing_score = 80`, `is_mandatory = true`
- Employee `emp-001` enrolled in the course

Test Steps:

Step	Action	Expected Result
1	<code>POST /lms/courses/course-compliance/complete</code> with <code>{ "score": 65 }</code>	HTTP 422 — <code>{ "error": "BELOW_PASSING_SCORE", "message": "Minimum passing score is 80. Your score: 65" }</code>
2	Inspect <code>course_enrollments.status</code>	<code>"in_progress"</code> — not changed to <code>"completed"</code>
3	Inspect <code>course_enrollments.status</code>	<code>"failed"</code> is NOT set yet — only set after explicit exhaustion of attempts
4	<code>POST /lms/courses/course-compliance/complete</code> with <code>{ "score": 80 }</code> (exactly at boundary)	HTTP 200 — <code>{ "status": "completed" }</code> — boundary score accepted
5	Verify HR dashboard	<code>emp-001</code> no longer shows as overdue for <code>course-compliance</code>
6	<code>GET /lms/assignments/all</code> filtered for <code>emp-001</code>	Mandatory assignment shows <code>status = "completed"</code>

Expected Result: Score below passing threshold blocked; exact boundary score accepted; completion reflects in assignment tracking.

Postconditions:

- Enrollment (status = "completed") only after passing score

Actual Result: _____

Pass / Fail: _____

LMS-003

Field	Value
Test Case ID	LMS-003
Title	Certification expiry alert sent before expiry date
Module	Learning & Development
Priority	P1
Severity	High
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Certification type (AWS-SAA) with (validity_months = 24)
- Employee (emp-001) holds certification issued (2023-06-01), expiry (2025-06-01)
- Current date is 31 days before expiry (2025-05-01)

Test Steps:

Step	Action	Expected Result
1	Trigger certification expiry check cron job	Job runs without error
2	Verify notification in <code>notifications</code> table for <code>emp-001</code>	Notification exists: <code>type = "certification.expiry_soon"</code> , body includes certification name and expiry date
3	Verify notification in <code>notifications</code> table for <code>emp-001</code> 's manager	Manager also notified about their report's expiring certification
4	<code>GET /certifications/expiring?days=30</code> (HR view)	Returns <code>emp-001</code> 's <code>AWS-SAA</code> in the results
5	Set current date to 8 days before expiry (2025-05-24); re-run cron	Second reminder notification sent (7-day alert)
6	Set current date to 1 day after expiry (2025-06-02); re-run cron	Certification marked as expired in <code>employee_certifications</code> ; escalation alert sent to HR

Expected Result: Reminders sent at 30 days and 7 days before expiry; certification flagged expired on the day after expiry date.

Postconditions:

- `employee_certifications.expiry_date` unchanged (reference date stays correct)
- Alert logs retained for audit

Actual Result: _____

Pass / Fail: _____

LMS-004

Field	Value
Test Case ID	LMS-004
Title	Skills matrix gap analysis identifies missing role requirements
Module	Learning & Development
Priority	P2

Field	Value
Severity	Medium
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Job title `Senior Engineer` requires skills: `Python (level 4)`, `System Design (level 3)`, `SQL (level 3)`
- Employee `emp-001` (Senior Engineer) has: `Python (level 2)`, `SQL (level 4)` — missing System Design entirely

Test Steps:

Step	Action	Expected Result
1	<code>GET /skills/gaps/emp-001</code>	HTTP 200 — gap analysis returned
2	Inspect <code>gaps</code> array	Contains: <code>{ "skill": "Python", "required": 4, "current": 2, "gap": 2 }</code> and <code>{ "skill": "System Design", "required": 3, "current": 0, "gap": 3 }</code>
3	Inspect <code>met</code> array	Contains: <code>{ "skill": "SQL", "required": 3, "current": 4, "surplus": 1 }</code>
4	<code>GET /skills/matrix?department=engineering</code>	Returns heatmap data: skills as columns, employees as rows, proficiency levels as cell values
5	Verify heatmap data includes <code>emp-001</code>	Row for <code>emp-001</code> shows correct proficiency levels
6	Verify courses recommended to close gaps	Response includes courses tagged with <code>Python</code> and <code>System Design</code> skills

Expected Result: Gap analysis identifies deficient and missing skills; SQL surplus correctly identified; course recommendations relevant to gaps.

Postconditions:

- No data modified — this is a read-only analytical query

Actual Result: _____

Pass / Fail: _____

Module 15 — Benefits & Compensation [V2]

BEN-001

Field	Value
Test Case ID	BEN-001
Title	Benefits enrollment — employee can enroll in one plan per benefit type
Module	Benefits & Compensation
Priority	P1
Severity	High
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Two active health plans exist: [plan-health-basic] and [plan-health-premium]
- Employee [emp-001] is authenticated
- Enrollment window is open

Test Steps:

Step	Action	Expected Result
1	<code>POST /benefits/enrollments</code> with <code>{ "planId": "plan-health-basic", "enrollmentDate": "2025-07-01" }</code>	HTTP 201 — enrolled in basic plan
2	Inspect <code>benefit_enrollments</code>	Row with <code>status = "active"</code> , <code>plan_id = "plan-health-basic"</code>
3	<code>POST /benefits/enrollments</code> with <code>{ "planId": "plan-health-premium", "enrollmentDate": "2025-07-01" }</code> (second health plan)	HTTP 409 — <code>{ "error": "DUPLICATE_ENROLLMENT", "message": "Already enrolled in a health benefit plan" }</code>
4	<code>DELETE /benefits/enrollments/:id</code> (unenroll from basic)	HTTP 200; <code>end_date</code> set; <code>status = "terminated"</code>
5	<code>POST /benefits/enrollments</code> with <code>plan-health-premium</code> after unenrolling	HTTP 201 — premium enrollment succeeds
6	Add dependents: <code>PATCH /benefits/enrollments/:id</code> with <code>{ "dependents": [{ "name": "Jane Doe", "relationship": "Spouse", "dateOfBirth": "1990-01-15" }] }</code>	HTTP 200 — dependent added to JSONB field
7	Verify in <code>audit_logs</code>	Enrollment and unenrollment logged with <code>action = "create"</code> and <code>action = "update"</code>

Expected Result: One enrollment per benefit type enforced; unenroll allows switching; dependents saved; audit trail complete.

Postconditions:

- `benefit_enrollments` contains active premium enrollment with one dependent
- Previous basic enrollment `status = "terminated"`

Actual Result: _____

Pass / Fail: _____

BEN-002

Field	Value
Test Case ID	BEN-002
Title	Bonus cycle — manager input restricted within approval workflow
Module	Benefits & Compensation
Priority	P1
Severity	High
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Bonus cycle `bonus-2025` in `planning` status; budget = 500,000
- Manager `mgr-001` has 3 direct reports: `emp-a`, `emp-b`, `emp-c`

Test Steps:

Step	Action	Expected Result
1	<code>POST /compensation/bonus-cycles/bonus-2025/open</code> (HR action)	HTTP 200; <code>status = "open"</code>
2	Manager <code>mgr-001</code> sets allocation for <code>emp-a</code> : <code>POST /compensation/bonus-cycles/bonus-2025/allocations</code> with <code>{ "employeeId": "emp-a", "targetPct": 15, "recommendedAmount": 9000 }</code>	HTTP 201
3	Manager attempts to set allocation for <code>emp-d</code> (not their direct report)	HTTP 403 — <code>{ "error": "SCOPE_VIOLATION", "message": "You can only set allocations for your direct reports" }</code>
4	Set allocations for <code>emp-b</code> (target 20% = 12,000) and <code>emp-c</code> (target 10% = 6,000)	HTTP 201 for both
5	Verify total recommended = 27,000; budget utilization = 27,000 / 500,000 = 5.4%	Budget utilization correct
6	<code>POST /compensation/bonus-cycles/bonus-2025/approve</code> (HR/Finance action)	HTTP 200; <code>status = "approved"</code>
7	Attempt manager allocation change after approval	HTTP 422 — <code>{ "error": "CYCLE_LOCKED", "message": "Allocations cannot be modified on an approved cycle" }</code>
8	<code>POST /compensation/bonus-cycles/bonus-2025/disburse</code>	HTTP 200; <code>status = "disbursed"</code>
9	Verify <code>bonus_allocations</code>	All 3 rows have <code>approved_amount</code> set and <code>approved_by_id</code> populated

Expected Result: Manager can only allocate for direct reports; cycle locks on approval; disbursement finalizes all amounts.

Postconditions:

- All 3 allocations disbursed; cycle `status = "disbursed"`

Actual Result: _____

Pass / Fail: _____

BEN-003

Field	Value
Test Case ID	BEN-003
Title	Total compensation statement reflects all components accurately
Module	Benefits & Compensation
Priority	P1
Severity	Medium
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Employee emp-001 has: ctc_annual = 120,000; enrolled in health plan (employer contribution = 2,400/yr); approved bonus = 18,000; equity grant of 1,000 RSUs at current valuation of \$10/share

Test Steps:

Step	Action	Expected Result
1	<code>GET /compensation/statement/emp-001</code>	HTTP 200 — statement returned
2	Verify <code>base_salary = 120,000</code>	Matches <code>employee_salaries.ctc_annual</code>
3	Verify <code>benefits_value = 2,400</code>	Employer health contribution included
4	Verify <code>bonus_paid = 18,000</code>	From approved <code>bonus_allocations</code>
5	Verify <code>equity_value = 10,000</code>	1,000 RSUs × \$10 current valuation
6	Verify <code>total = 150,400</code>	Sum of all four components
7	Employee accesses same endpoint via <code>GET /compensation/statement/emp-001</code> with employee token	HTTP 200 — employee can view their own statement
8	Employee accesses <code>GET /compensation/statement/emp-002</code> with employee token	HTTP 403 — cannot view another employee's statement

Expected Result: All compensation components summed correctly; access control enforced (self-only for employees).

Postconditions:

- Read-only operation; no data modified

Actual Result: _____

Pass / Fail: _____

Module 16 — Surveys & Engagement [V2]

SRV-001

Field	Value
Test Case ID	SRV-001
Title	Anonymous survey — employee identity not linked to responses
Module	Surveys & Engagement

Field	Value
Priority	P1
Severity	Critical
Type	Security
Phase	V2
Status	Active

Preconditions:

- Survey `survey-pulse-q1` exists with `is_anonymous = TRUE`, `status = "active"`
- 5 employees assigned via `survey_assignments`

Test Steps:

Step	Action	Expected Result
1	Employee <code>emp-001</code> submits <code>POST /surveys/survey-pulse-q1/respond</code> with responses	HTTP 200 — response accepted
2	Inspect <code>survey_responses</code> table for the submitted row	<code>employee_id = NULL</code> — identity not stored
3	HR sends <code>GET /surveys/survey-pulse-q1/results</code>	Aggregated results returned (e.g., average rating = 3.8); no individual identity revealed
4	HR attempts to cross-reference response timestamp with <code>survey_assignments.sent_at</code> to identify respondent	No unique timestamp stored per response that would enable identification
5	Verify <code>survey_assignments</code> for <code>emp-001</code>	<code>reminded_at</code> updated to show they have responded (completion tracking preserved separately from response content)
6	Employee attempts <code>POST /surveys/survey-pulse-q1/respond</code> again	HTTP 409 — <code>{ "error": "ALREADY_RESPONDED", "message": "You have already submitted a response for this survey" }</code>

Expected Result: `employee_id` NULL in responses table; HR sees only aggregated data; re-submission blocked; completion tracking doesn't break anonymity.

Postconditions:

- `survey_responses` row with `employee_id = NULL`
- `survey_assignments` row updated to reflect completion (without linking to specific response)

Actual Result: _____

Pass / Fail: _____

SRV-002

Field	Value
Test Case ID	SRV-002
Title	Survey lifecycle — draft cannot receive responses; closed survey rejects new submissions
Module	Surveys & Engagement
Priority	P1
Severity	High
Type	Negative
Phase	V2
Status	Active

Preconditions:

- Survey `survey-draft` in `draft` status
- Survey `survey-closed` in `closed` status
- Employee `emp-001` assigned to both surveys

Test Steps:

Step	Action	Expected Result
1	<code>POST /surveys/survey-draft/respond</code> with valid responses	HTTP 422 — <code>{ "error": "SURVEY_NOT_ACTIVE", "message": "Survey is not currently open for responses" }</code>
2	<code>POST /surveys/survey-closed/respond</code> with valid responses	HTTP 422 — <code>{ "error": "SURVEY_CLOSED", "message": "This survey is no longer accepting responses" }</code>
3	HR publishes draft survey: <code>POST /surveys/survey-draft/publish</code>	HTTP 200; <code>status = "active"</code>
4	<code>POST /surveys/survey-draft/respond</code> after publishing	HTTP 200 — response now accepted
5	HR closes survey: <code>POST /surveys/survey-draft/close</code>	HTTP 200; <code>status = "closed"</code>
6	<code>POST /surveys/survey-draft/respond</code> after closing	HTTP 422 — <code>{ "error": "SURVEY_CLOSED" }</code>

Expected Result: Only `active` surveys accept responses; state transitions correctly gate submission access.

Postconditions:

- Survey `status = "closed"`; all submitted responses retained

Actual Result: _____

Pass / Fail: _____

Module 17 — Offboarding [V2]

OFF-001

Field	Value
Test Case ID	OFF-001
Title	Exit request workflow — submission through approval with full audit trail
Module	Offboarding

Field	Value
Priority	P1
Severity	High
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Employee `emp-001` is active with `notice_period_days = 30`
- HR Manager `hr-001` is authenticated

Test Steps:

Step	Action	Expected Result
1	Employee submits <code>POST /offboarding</code> with <code>{ "resignationDate": "2025-06-01", "exitType": "voluntary", "reasonCategory": "better_opportunity" }</code>	HTTP 201; <code>status = "pending"</code>
2	System computes <code>last_working_date</code>	<code>2025-07-01</code> (resignation + 30 days notice)
3	HR sends <code>POST /offboarding/:id/approve</code>	HTTP 200; <code>status = "approved"</code> , <code>approved_by_id</code> populated
4	Verify <code>employee.status</code>	Remains <code>"active"</code> until <code>last_working_date</code> — not terminated prematurely
5	Verify exit checklist auto-created	<code>exit_checklists</code> rows created for IT, Finance, HR categories
6	IT admin completes task: <code>PATCH /offboarding/:id/checklist/:taskId/complete</code>	HTTP 200; <code>completed_at</code> set
7	<code>GET /offboarding/:id/checklist</code>	Shows mix of completed and pending tasks
8	On <code>last_working_date</code> , system job sets <code>employee.status = "terminated"</code> and <code>employee.last_working_date</code>	Employee terminated automatically
9	Verify <code>audit_logs</code>	Entries for: exit request created, exit approved, checklist tasks completed, employee terminated

Expected Result: Full offboarding lifecycle; employee active until LWD; checklist tracked; termination automated; audit complete.

Postconditions:

- `employees.status = "terminated"`, `deleted_at = NULL` (soft delete)
- All offboarding records retained

Actual Result: _____

Pass / Fail: _____

OFF-002

Field	Value
Test Case ID	OFF-002
Title	Exit interview responses stored and accessible only to HR
Module	Offboarding
Priority	P1
Severity	High
Type	Security
Phase	V2
Status	Active

Preconditions:

- Exit request `exit-001` in `approved` status for employee `emp-001`
- HR Manager `hr-001` is authenticated

Test Steps:

Step	Action	Expected Result
1	<code>POST /offboarding/exit-001/exit-interview</code> with <code>{ "scheduledAt": "2025-06-25T10:00:00Z", "interviewerId": "hr-001" }</code>	HTTP 201 — interview scheduled
2	HR submits interview responses: <code>PATCH /offboarding/exit-001/exit-interview/:interviewId</code> with structured responses and notes	HTTP 200; responses saved in <code>exit_interviews.responses</code> JSONB
3	<code>GET /offboarding/exit-001/exit-interview</code> using HR token	HTTP 200 — full responses returned including notes
4	<code>GET /offboarding/exit-001/exit-interview</code> using departing employee's token	HTTP 403 — <code>{ "error": "FORBIDDEN", "message": "Exit interview details are HR-only" }</code>
5	<code>GET /offboarding/exit-001/exit-interview</code> using the departing employee's manager token	HTTP 403 — manager does not have access to exit interview details
6	Verify in <code>audit_logs</code>	Access by HR logged; 403 attempts logged as <code>action = "attempt_failed"</code>

Expected Result: Exit interview accessible only to HR role; employee and manager receive 403; all access attempts audited.

Postconditions:

- `exit_interviews.completed_at` populated after step 2
- `exit_interviews.responses` persisted as JSONB

Actual Result: _____

Pass / Fail: _____

OFF-003

Field	Value
Test Case ID	OFF-003
Title	Notice waiver — last working date set to resignation date when notice waived

Field	Value
Module	Offboarding
Priority	P2
Severity	Medium
Type	Functional
Phase	V2
Status	Active

Preconditions:

- Employee emp-001 has notice_period_days = 60
- HR Manager authenticated

Test Steps:

Step	Action	Expected Result
1	POST /offboarding with { "resignationDate": "2025-06-01", "exitType": "involuntary", "noticeWaived": true }	HTTP 201
2	Inspect last_working_date in response	"2025-06-01" — equals resignation date since notice is waived, not 2025-08-01
3	POST /offboarding/:id/approve	HTTP 200
4	Verify in exit_requests table	notice_waived = TRUE, last_working_date = "2025-06-01", resignation_date = "2025-06-01"
5	Attempt POST /offboarding for same employee while first exit request is pending	HTTP 409 — { "error": "DUPLICATE_EXIT_REQUEST", "message": "An exit request is already in progress for this employee" }

Expected Result: Notice waiver sets LWD to resignation date; duplicate exit request for same employee blocked.

Postconditions:

- `exit_requests.notice_waived = TRUE`
- `exit_requests.last_working_date = exit_requests.resignation_date`

Actual Result: _____

Pass / Fail: _____

Module 18 — Notifications `[V1]`

NOTIF-001

Field	Value
Test Case ID	NOTIF-001
Title	Notification delivered in-app and via email on leave approval
Module	Notifications
Priority	P1
Severity	High
Type	Integration
Phase	V1
Status	Active

Preconditions:

- Leave request `1r-001` in `pending` status for `emp-001`
- Mailhog running in local/staging environment to capture outbound emails

Test Steps:

Step	Action	Expected Result
1	Manager approves: <code>POST /leave/requests/lr-001/approve</code>	HTTP 200 — leave approved
2	<code>GET /notifications</code> using <code>emp-001</code> token	Notification with <code>type = "leave.approved"</code> appears in inbox
3	Inspect notification	<code>title</code> contains employee name; <code>data.resourceId = "lr-001"</code> ; <code>read_at = NULL</code>
4	Inspect Mailhog/email queue	Email sent to <code>emp-001.email</code> with subject containing "Leave Request Approved"
5	Inspect email body	Contains: employee name, leave type, approved dates, approver name
6	Employee marks notification read: <code>PATCH /notifications/:id/read</code>	HTTP 200; <code>read_at</code> populated
7	<code>GET /notifications/unread-count</code>	Count decremented by 1
8	<code>POST /notifications/read-all</code>	HTTP 200; all unread notifications for <code>emp-001</code> marked read

Expected Result: In-app notification created and readable; email sent with correct content; read/unread state managed correctly.

Postconditions:

- `notifications.read_at` populated for `emp-001`'s leave approval notification
- Email record in Mailhog with correct recipient and subject

Actual Result: _____

Pass / Fail: _____

NOTIF-002

Field	Value
Test Case ID	NOTIF-002
Title	Notification template variable substitution renders correctly

Field	Value
Module	Notifications
Priority	P1
Severity	Medium
Type	Functional
Phase	V1
Status	Active

Preconditions:

- Notification template for `payslip.ready` exists with body: `"Hi {{employee_name}}, your payslip for {{period}} is ready. Net pay: {{currency}} {{net_pay}}."`
- Payslip generated for `emp-001` for period `January 2025`, net pay `57,000 BDT`

Test Steps:

Step	Action	Expected Result
1	Trigger payslip-ready notification for <code>emp-001</code> (via payslip generation job completion)	Notification created in DB
2	<code>GET /notifications</code> using <code>emp-001</code> token	Notification returned
3	Inspect notification <code>body</code>	<code>"Hi John Doe, your payslip for January 2025 is ready. Net pay: BDT 57,000."</code> — all variables substituted
4	Verify no raw template variables remain	Body does NOT contain <code>{{employee_name}}</code> , <code>{{period}}</code> , etc.
5	Verify email body (Mailhog)	Same rendered content in email
6	Test company-level override template: create <code>payslip.ready</code> template for Company A with custom branding text	Company A employees receive the custom template; Company B receives the system default

Expected Result: All template variables substituted; no raw placeholders in output; company-level override applied for Company A.

Postconditions:

- `notifications.body` contains rendered text with no `{{}}` placeholders

Actual Result: _____

Pass / Fail: _____

Module 19 — API Security & Input Validation [V1]

SEC-001

Field	Value
Test Case ID	SEC-001
Title	SQL injection attempt returns validation error without executing
Module	API Security
Priority	P0
Severity	Critical
Type	Security
Phase	V1
Status	Active

Preconditions:

- Any authenticated API user

Test Steps:

Step	Action	Expected Result
1	<code>GET /employees?search='; DROP TABLE employees; --</code>	HTTP 400 — Zod validation error OR HTTP 200 with no results (safely parameterized query)
2	Verify <code>employees</code> table after request	Table still exists; no data deleted
3	<code>POST /employees</code> with <code>firstName: "Robert'); DROP TABLE employees; --"</code>	HTTP 201 — employee created with the string stored literally as the first name
4	Verify <code>employees</code> table	Employee exists with <code>first_name = "Robert');</code> <code>DROP TABLE employees; --</code> stored as text, not executed
5	Verify query in DB logs	Parameterized query used; injection string treated as data value

Expected Result: Parameterized queries prevent SQL injection; malicious input stored as literal text; no database modification.

Postconditions:

- All tables intact
- Injected test employee cleaned up after test

Actual Result: _____

Pass / Fail: _____

SEC-002

Field	Value
Test Case ID	SEC-002
Title	Request body size limit blocks oversized payloads
Module	API Security
Priority	P1
Severity	High
Type	Security

Field	Value
Phase	V1
Status	Active

Preconditions:

- API configured with max request body size of 5MB for JSON endpoints, 20MB for file uploads

Test Steps:

Step	Action	Expected Result
1	Send <code>POST /employees</code> with a JSON body of 6MB (padded with a large <code>customFields</code> object)	HTTP 413 — <code>{ "error": "PAYLOAD_TOO_LARGE", "message": "Request body exceeds the 5MB limit" }</code>
2	Send <code>POST /employees/:id/documents</code> with a 19MB PDF file	HTTP 200 — upload accepted
3	Send <code>POST /employees/:id/documents</code> with a 21MB PDF file	HTTP 413 — file too large
4	Send <code>POST /employees</code> with a valid 1KB JSON body	HTTP 201 — accepted normally
5	Verify no partial data written to DB in steps 1 and 3	No employee or document record created from rejected requests

Expected Result: Oversized JSON body and file uploads rejected with 413; valid payloads accepted; no partial writes.

Postconditions:

- No database records created from rejected requests

Actual Result: _____

Pass / Fail: _____

SEC-003

Field	Value
Test Case ID	SEC-003
Title	Security response headers present on all API responses
Module	API Security
Priority	P1
Severity	High
Type	Security
Phase	V1
Status	Active

Preconditions:

- API running with Helmet middleware configured

Test Steps:

Step	Action	Expected Result
1	Send <code>GET /employees</code> and inspect response headers	<code>X-Content-Type-Options: nosniff</code> present
2	Inspect <code>X-Frame-Options</code> header	<code>DENY</code> or <code>SAMEORIGIN</code> present
3	Inspect <code>Strict-Transport-Security</code> header	<code>max-age=31536000; includeSubDomains</code> present
4	Inspect <code>Content-Security-Policy</code> header	Restrictive CSP policy present
5	Inspect <code>X-Powered-By</code> header	Header NOT present — server fingerprinting disabled
6	Send <code>GET /employees</code> via HTTP (not HTTPS)	Redirect to HTTPS (301) or connection rejected
7	Inspect <code>Referrer-Policy</code> header	<code>strict-origin-when-cross-origin</code> or stricter

Expected Result: All required security headers present; X-Powered-By absent; HTTP redirects to HTTPS.

Postconditions:

- No data modified — header inspection only

Actual Result: _____

Pass / Fail: _____

Regression Test Execution Log

Run #	Date	Build / Release	Tester	Environment	Total	Passed	Failed	Blocked	Pass Rate
1				Staging					
2				Staging					
3				Staging					
4				Production					

Defect log template

When a test case fails, log the defect using this format:

Field	Value
Defect ID	DEF-YYYYMMDD-NNN
Test Case ID	e.g. PAY-001
Title	Brief description of the defect
Severity	Critical / High / Medium / Low
Priority	P0 / P1 / P2 / P3
Steps to reproduce	Exact steps that triggered the failure
Expected result	What should have happened
Actual result	What actually happened
Environment	Staging / Production + build version
Attachments	Screenshot, API response, DB query result
Assigned to	Developer name
Status	Open / In Progress / Fixed / Verified / Closed
Regression link	Link to the bug-driven regression test added for this defect

Test case traceability matrix

Test Case ID	Feature / Requirement	Module	Sprint	Priority
AUTH-001	FR-AUTH-01: Valid login issues JWT	Authentication	Sprint 1	P0
AUTH-002	FR-AUTH-02: Invalid credentials rejected	Authentication	Sprint 1	P0
AUTH-003	FR-AUTH-03: Refresh token rotation	Authentication	Sprint 1	P0
AUTH-004	FR-AUTH-04: Expired token rejected	Authentication	Sprint 1	P0
AUTH-005	NFR-SEC-01: Cross-tenant isolation	Multi-tenancy	Sprint 1	P0
AUTH-006	NFR-SEC-02: Brute-force protection	Authentication	Sprint 1	P1
AUTH-007	FR-RBAC-01: Role-based access control	RBAC	Sprint 1	P0
AUTH-008	FR-AUTH-05: Password reset single-use	Authentication	Sprint 1	P1

Test Case ID	Feature / Requirement	Module	Sprint	Priority
EMP-001	FR-EMP-01: Create employee	Employee	Sprint 2	P0
EMP-002	FR-EMP-02: Prevent duplicate employee number	Employee	Sprint 2	P0
EMP-003	FR-EMP-03: Terminate employee	Employee	Sprint 2	P0
EMP-004	FR-EMP-04: Soft delete preserves data	Employee	Sprint 2	P0
EMP-005	NFR-SEC-03: PII field encryption	Employee / Security	Sprint 2	P0
EMP-006	FR-EMP-05: Bulk CSV import	Employee	Sprint 2	P1
LVE-001	FR-LVE-01: Monthly leave accrual	Leave	Sprint 3	P0
LVE-002	FR-LVE-02: Insufficient balance rejection	Leave	Sprint 3	P0
LVE-003	FR-LVE-03: Approval updates balance	Leave	Sprint 3	P0
LVE-004	FR-LVE-04: Balance cannot go negative	Leave	Sprint 3	P0
LVE-005	FR-LVE-05: Holiday conflict detection	Leave	Sprint 3	P1
LVE-006	FR-LVE-06: Year-end carry-forward cap	Leave	Sprint 3	P1
ATT-001	FR-ATT-01: Clock-in / clock-out	Attendance	Sprint 3	P0
ATT-002	FR-ATT-02: Prevent double clock-in	Attendance	Sprint 3	P0
ATT-003	FR-ATT-03: Manual correction audit	Attendance	Sprint 3	P0
PAY-001	FR-PAY-01: Net pay computation	Payroll	Sprint 4	P0
PAY-002	FR-PAY-02: LOP deduction	Payroll	Sprint 4	P0
PAY-003	FR-PAY-03: Cycle state machine	Payroll	Sprint 4	P0
PAY-004	FR-PAY-04: Payslip access control	Payroll	Sprint 4	P0
PAY-005	FR-PAY-05: Bank file accuracy	Payroll	Sprint 4	P1
DOC-001	FR-DOC-01: Policy acknowledgement	Documents	Sprint 5	P0
DOC-002	FR-DOC-02: eSign hash integrity	Documents	Sprint 5	P0
AUD-001	NFR-AUD-01: All mutations audited	Audit	Sprint 5	P0
AUD-002	NFR-AUD-02: Audit log append-only	Audit	Sprint 5	P0

Test Case ID	Feature / Requirement	Module	Sprint	Priority
AUD-003	NFR-AUD-03: Audit non-blocking	Audit	Sprint 5	P1
REC-001	FR-REC-01: Pipeline stage transitions	Recruitment	Sprint 7	P0
REC-002	FR-REC-02: Offer acceptance creates employee	Recruitment	Sprint 7	P0
PERF-001	FR-PERF-01: OKR goal check-ins	Performance	Sprint 8	P1
PERF-002	FR-PERF-02: 360 anonymity + calibration	Performance	Sprint 8	P1
RPT-001	FR-RPT-01: Headcount report accuracy	Reporting	Sprint 6	P1
RPT-002	FR-RPT-02: Async XLSX export	Reporting	Sprint 6	P1
WH-001	FR-INT-01: Webhook HMAC signature	Webhooks	Sprint 12	P1
WH-002	FR-INT-02: Webhook auto-disable	Webhooks	Sprint 12	P1
MT-001	NFR-SEC-04: Tenant data isolation (lists)	Multi-tenancy	Sprint 1	P0
MT-002	NFR-SEC-05: Tenant isolation (by ID)	Multi-tenancy	Sprint 1	P0
PERF-LOAD-001	NFR-PERF-01: Employee list p95 < 120ms	Performance	Sprint 6	P1
PERF-LOAD-002	NFR-PERF-02: Payroll 500 emp < 30s	Performance	Sprint 6	P1
BOUND-001	FR-PAY-06: Zero salary edge case	Payroll / Boundary	Sprint 4	P2
BOUND-002	FR-PAY-07: Single-day hire proration	Payroll / Boundary	Sprint 4	P2
BOUND-003	FR-LVE-07: Weekend exclusion from leave count	Leave / Boundary	Sprint 3	P2
BOUND-004	FR-EMP-06: VARCHAR max length enforcement	Employee / Boundary	Sprint 2	P2
LMS-001	FR-LMS-01: Course enrollment and completion	Learning & Dev	Sprint 9	P1
LMS-002	FR-LMS-02: Mandatory course passing score	Learning & Dev	Sprint 9	P1
LMS-003	FR-LMS-03: Certification expiry alerts	Learning & Dev	Sprint 9	P1

Test Case ID	Feature / Requirement	Module	Sprint	Priority
LMS-004	FR-LMS-04: Skills gap analysis	Learning & Dev	Sprint 9	P2
BEN-001	FR-BEN-01: Benefits enrollment one-per-type	Benefits	Sprint 10	P1
BEN-002	FR-BEN-02: Bonus cycle manager input and approval	Benefits	Sprint 10	P1
BEN-003	FR-BEN-03: Total compensation statement accuracy	Benefits	Sprint 10	P1
SRV-001	FR-SRV-01: Anonymous survey identity protection	Surveys	Sprint 10	P1
SRV-002	FR-SRV-02: Survey lifecycle state gating	Surveys	Sprint 10	P1
OFF-001	FR-OFF-01: Exit request full workflow	Offboarding	Sprint 11	P1
OFF-002	FR-OFF-02: Exit interview HR-only access	Offboarding	Sprint 11	P1
OFF-003	FR-OFF-03: Notice waiver sets LWD correctly	Offboarding	Sprint 11	P2
NOTIF-001	FR-NOTIF-01: In-app and email notification delivery	Notifications	Sprint 5	P1
NOTIF-002	FR-NOTIF-02: Template variable substitution	Notifications	Sprint 5	P1
SEC-001	NFR-SEC-06: SQL injection prevention	API Security	Sprint 1	P0
SEC-002	NFR-SEC-07: Request body size limit	API Security	Sprint 1	P1
SEC-003	NFR-SEC-08: Security response headers	API Security	Sprint 1	P1